Note: this write-up was drafted with Claude (Anthropic), working from my code, git history and learning log. I reviewed and edited it.

1. Task

Input: a messy NZ vehicle string (e.g. `Toyota Surf - RZN130`). Output: a canonical make|model from the NZ Motor Vehicle Register, or `None`. `None` is a legitimate answer.

- **Purpose:** Matching parts that are listed under different model names so wreckers and buyers can transact the correct part.
- **Cost of errors:** A wrong vehicle is much worse than 'None'.

2. Data

- **Noisy side:** 1,721 distinct vehicle strings scraped from one parts retailer's catalogue. Semi-structured (`Make Model - Chassis/Variant Years`), so cleaner than free-text marketplace listings.
- **Canonical side:** NZTA Motor Vehicle Register (5.9M rows), aggregated to distinct make|model pairs and cut at fleet count ≥ 10 → 12,728 pairs covering 95% of the fleet. Long-tail pairs below the cut don't exist as possible answers.
- **Sampling:** the 200 gold strings were drawn uniformly over the 1,721 distinct strings, not weighted by how common a vehicle is.
- **Scope:** The result is claimed for the 1,721 Partsworld strings only. It isn't tested on other sources, including free-text marketplace listings. The 200 gold rows are a sample used to estimate performance on all 1,721, because hand-labelling every string would take too long.

3. Gold set

- 200 strings drawn from the corpus with a fixed seed. Split randomly over distinct strings: 100 dev / 100 held-out.
- One labeller (me). No second annotator; self-agreement not yet measured.
- Label = register make, model, and variant (variant only when the string contains one — 5 of 100 dev rows). A row is labelled `NONE` when no register vehicle fits (e.g. `Renault All Series`).
- Gold was labelled against the cut table (12,728 pairs), not the full register: the labelling tool only searches the cut table. A vehicle below the cut can only be labelled `NONE` or as another vehicle, so some `NONE` rows may be artefacts of the cut. All 99 non-`NONE` dev labels are in the cut table.
- Base rates: dev has 1 `NONE` row out of 100. Held-out base rates aren't counted yet, because counting them means reading held-out labels; they'll be reported with the final run.
- JDM vs export rule (applied during labelling, not written down at the time): if the cut table holds the name the string uses, gold uses that name (`Honda Inspire` → HONDA|INSPIRE). If it doesn't, gold uses the register's name for the same vehicle (`Toyota Surf` → TOYOTA|4RUNNER, `Mitsubishi Emmeraude` → MITSUBISHI|GALANT; neither SURF nor EMMERAUDE is in the cut table).
- Duplicate spellings: gold holds one spelling per vehicle (MAZDA|MX5). The other register spelling (MAZDA|MX-5) is scored wrong.
- Held-out rows were labelled on 09/09, before the mid-project rule change (19/09), and haven't been relabelled, so they follow the old rules. Checking which held-out rows the new rule would change means reading their labels, so it hasn't been done.
- The dev labels were revised once mid-project, under a new rule: model = parent line, variant = sub-model, each labelled only if the string contains it. Five rows changed. Earlier approaches were re-scored on the revised labels.
  - Model split from variant (key unchanged, variant added): `Ford XR6 - FG-FGX 2008-2016` was FORD|FALCON → FORD|FALCON, variant XR6.
  - BMW rows moved to the parent line (key changed): `BMW 3 Series E46 - 318i` was BMW|318 → BMW|3 Series, variant 318i.
- A second relabel the same day (two rows changed to match the alias table) was reverted within hours; those rows keep their original labels.

4. What counts as correct

- Score key is MAKE|MODEL, exact match, case-insensitive. Variant is stripped and scored separately on the 5 rows that have one.
- **Accuracy:** correct / 100. Abstention is the string `"NONE"`, and it counts as correct when gold is `NONE`. A Python `None` means no prediction and always counts as wrong.
- **For approaches that can abstain** (answer `NONE` instead of guessing): precision = of the rows it answered, how many were right. Coverage = how many of the 100 rows it answered at all. A correct `NONE` counts toward accuracy, but `NONE` isn't an answer, so it's left out of precision.
- Abstention rule: abstain on any tie. Confidence is 1 ÷ the number of tied candidate vehicles (1.0, 0.5, 0.33…). The fuzzy fallback could in principle give values in between, but none occurred on dev. The 0.9 cut-off is fixed, not tuned: any value between 0.5 and 1.0 gives the same result. Calibration check: are the untied rows right more often than the tied ones?
- Point estimates only so far; no intervals.

5. How the dev set is used

- Five approaches, all measured on the same 100 dev rows:
  1. Exact: candidates are register pairs whose make and model both appear in the string (case-insensitive substring). Longest model name wins, then highest fleet count, then alphabetical. So `Dodge Ram 1500` → RAM|1500, because "1500" is longer than "RAM".
  2. Fuzzy: rapidfuzz `fuzz.ratio`, scoring make and model separately against runs of up to 4 adjacent words, punctuation deleted. Both must score 95+ out of 100. Ties use approach 1's rule.
  3. Alias table + chassis codes: a table for the 40 register vehicles with the highest fleet count (39.6% of the fleet; 192 rows of chassis codes, JDM name and export name), mapping those names and codes (e.g. RZN130) to the register model. Names are tried first; the chassis code only when no name matches.
  4. LLM extraction: an LLM copies make, model and chassis code out of the string verbatim, then approach 3 does the matching. Details below.
  5. Abstention: approach 4's answer, or `NONE` when two or more candidate vehicles tie.
- The alias-table approach went through four iterations on those rows. The fuzzy threshold (95) was checked with a sweep on dev: 100, 95 and 80 gave identical predictions.
- Alias table history: the 40 vehicles were picked by fleet-count rank, not from dev errors. The table file was written once (19/09), after the dev set had been labelled and read through Reps 1–2, and hasn't been edited since. The four iterations changed how the code uses it (chassis codes, name priority, dropping alias names that are themselves register models), each in response to dev errors. Number of held-out rows in those 40 families: not yet counted, since counting means reading the held-out set. It will be counted during the held-out relabel and reported with the final run.
- LLM details: model `claude-opus-5` (fallback `claude-opus-4-8`, used 0 times), effort low, max 1,024 output tokens, run 29/09. Input: the title only. Output: JSON schema with three required string fields: make, model, chassis. System prompt: "You read one vehicle listing title from a New Zealand car parts catalogue and copy three fields out of it: the make, the model and the chassis code. Copy each field exactly as the title writes it. Do not translate a name into another market's name, correct its spelling, or replace it with a parent model or a variant. Leave a field as an empty string when the title does not state it. Year ranges, generation notes and trim descriptions go in none of the fields."
- Before every change I read all errors, categorise them, and write a numeric prediction; then measure.
- The LLM approach runs once with cached replies (no temperature control available), so run-to-run variance isn't measured.
- The held-out 100 rows are kept untouched while I build. I looked at them once to label them, will relabel them once to the current rules before the final run (see section 6), and I'll score the finished system on them once at the end, without changing anything after seeing that result.

6. Weaknesses I already know about

- No confidence intervals; n = 100.

- Split isn't independent: strings from the same make/model family land on both sides.
- Single labeller — my label-noise ceiling is unknown.
- Label contamination: the mid-project relabel (section 3) was triggered by reading errors, and it moved gold towards what the system already predicted — 3 previously wrong rows became correct, none went the other way. The fix was a rule applied to every row, not row-by-row edits, but it was still decided after seeing predictions.
- The register holds punctuated and unpunctuated spellings as separate models (`MX-5` / `MX5`); once normalised, which one wins is a tie-break, not a match.
- The tied (abstained) group holds 8 rows — too few to judge calibration.
- The harness disagrees with itself and with the package. `resolve()` documents Python `None` as a first-class outcome, but accuracy scores `None` as wrong, while precision/coverage treat it the same as `"NONE"`. None of the five approaches returns Python `None` today, so no reported number is affected yet.
- Held-out labels follow the pre-19/09 rules, so as they stand the final score would be measured against a different standard from dev. Before the Week 4 run I'll relabel the held-out set to the current rules (model/variant and JDM/export, as written in section 3), without running any approach on it first. That is an extra look at the held-out set, and it's logged as one.
- No run-to-run variance for the LLM approaches: each title was sent once and the reply cached, so approaches 4 and 5 are scored on a single sample of LLM output. I don't know how much the score would move on a re-run.

7. Examples

Gold rows (dev):

Input	Make	Model	Variant
Honda Legend - KB1	HONDA	LEGEND	
Toyota Sprinter - AE110	TOYOTA	SPRINTER	
Mitsubishi Lancer - CK4A 1994-1998	MITSUBISHI	LANCER	
Toyota Surf - RZN130	TOYOTA	4RUNNER	
Renault Renault All Series - All Series	NONE		

Harness output format. Every approach gets a score line; the abstaining approach gets two more:

```
exact_match: score=c/100 (x.xx)  variant=v/5 (x.xx)
...
rep5_abstain: score=c/100 (x.xx)  variant=v/5 (x.xx)
rep5_abstain: precision=c/a (x.xx)  coverage=a/100 (x.xx)
rep5_abstain: rep4_llm_extract errors turned to NONE=n/e
```

- `score`: accuracy on MAKE|MODEL. `variant`: variant accuracy on the 5 rows with a gold variant (counts only when the key is also right).
- `precision` / `coverage`: as defined in section 4.
- `errors turned to NONE`: how many of approach 4's wrong answers approach 5 replaced with `NONE`.
- A `stub` approach that returns Python `None` (no prediction) for every row is run alongside. It must score 0. That checks the harness counts a missing prediction as wrong, rather than letting it match a gold `NONE`.